SQL: Joins and Subqueries

Databases and Software Architecture – Aalborg University

Claude, a large language model developed by Anthropic, was used to assist with structuring and grammar checking of this material. All generated text was critically reviewed, edited, and verified by the author.

Joins and Subqueries

Last lecture asked questions of **one** table at a time. But real questions rarely stay inside one table – “which members currently have an overdue book, and what is it called?” touches `Member`, `Loan`, **and** `Book` at once. Today: combining rows across tables with `JOIN`, and nesting one query inside another with subqueries.

Agenda

Today: motivation, all the JOIN types, a dedicated look at a classic JOIN trap, subqueries in `WHERE`, a break, subqueries in `FROM`/`SELECT`, a practical exercise, and a CTE preview.

- Motivation: why one table is never enough
- `JOIN` types: `INNER`, `LEFT`, `RIGHT`, `FULL OUTER`, self-join, `CROSS JOIN`, `ON` vs `USING`
- The `ON` vs `WHERE` trap – a dedicated deep dive
- Subqueries in `WHERE`: `IN`, `NOT IN`, `EXISTS`, `NOT EXISTS`, single-value comparison
- Break
- Subqueries in `FROM` and `SELECT`, correlated subqueries
- Practical exercise: six complex queries
- CTEs (`WITH`) – a brief preview

Motivation

Recap: The Library Schema

The same seven tables as the last two lectures, unchanged: `Author`, `Book`, `Member`, `BookGenre`, `Copy`, `BookAuthor`, `Loan`. One new column today: `Member.referred_by`, a self-reference used for one example.

- `Book(book_id, title, isbn, publication_year)`
- `Author(author_id, first_name, last_name, birth_year)`
- `Member(member_id, first_name, last_name, email, join_date, referred_by)`
- `Loan(loan_id, member_id, book_id, copy_number, loan_date, due_date, return_date)`
- Plus `BookGenre`, `Copy`, `BookAuthor` – junction/weak-entity tables linking the above

The Problem: Data Is Spread Across Tables

"Which members currently have an overdue book, and what is it called?"

- `Member` alone: has names, but no idea who borrowed what
- `Loan` alone: has `member_id` and `book_id`, but no names and no titles
- `Book` alone: has titles, but no idea who has which copy

No single table answers the question. The answer exists only in the **combination** of rows across tables – that combination is exactly what `JOIN` computes.

Today’s Question: How Do We Combine Rows Across Tables?

- `JOIN` – glue rows from two (or more) tables together, row by row, based on a condition
- Subqueries – nest one complete `SELECT` inside another, to filter, compute, or supply a temporary table
- Both are ways of answering questions that no single table can answer alone

By the end of today: overdue-loan lookups, multi-table reports, and existence checks like “which members have returned every book they ever borrowed.”

JOIN Types

The JOIN Clause: General Shape

```
SELECT columns
FROM table1
JOIN table2 ON table1.column = table2.column;
```

- JOIN sits inside FROM – conceptually, it builds one combined table before WHERE/GROUP BY ever run
- The ON condition says **how** rows from the two tables line up – almost always a foreign-key match
- Several JOIN **types** exist, differing only in what happens to rows that don't find a match

INNER JOIN: Only Matching Rows

```
SELECT b.title, c.copy_number, c.status
FROM Book b
JOIN Copy c ON b.book_id = c.book_id
ORDER BY b.title, c.copy_number
LIMIT 5;
```

title	copy_number	status
2001: A Space Odyssey	1	available
2001: A Space Odyssey	2	available
A Farewell to Arms	1	available
A Farewell to Arms	2	available
Americanah	1	available

Plain `JOIN` means `INNER JOIN` – a row survives only if **both** sides have a match. Every book here has at least one copy (80 books, 157 copies total), so nothing is lost – next slide shows what happens when that is not true.

LEFT JOIN: Keep Every Row From the Left

```sql
SELECT b.book_id, b.title, l.loan_id
FROM Book b
LEFT JOIN Loan l ON b.book_id = l.book_id;
```

Loan has 258 rows referencing books – but only 78 **distinct** books were ever borrowed, out of 80 total.

Join type	Row count
INNER JOIN (Book / Loan)	258
LEFT JOIN (Book / Loan)	260

The 2 extra rows are the books with **no** matching loan – LEFT JOIN keeps them anyway, with l.loan_id padded to NULL, instead of silently dropping them the way INNER JOIN does.

`LEFT JOIN`: The Two Extra Rows

```
SELECT b.book_id, b.title, l.loan_id
FROM Book b LEFT JOIN Loan l ON b.book_id = l.book_id
WHERE l.loan_id IS NULL;
```

book_id	title	loan_id
6	Guards! Guards!	NULL
53	Beloved	NULL

“All books, and their loans if they have any” – `LEFT JOIN` is the natural shape for “show me everything on the left, annotated with matching data on the right when it exists.”

RIGHT JOIN: Keep Every Row From the Right

The mirror image of LEFT JOIN – keeps every row from the **second** table, padding the first with NULL where nothing matches.

```
SELECT b.book_id, b.title, l.loan_id
FROM Loan l RIGHT Book b JOIN  ON b.book_id = l.book_id;
-- identical result to:
SELECT b.book_id, b.title, l.loan_id
FROM Book b LEFT JOIN Loan l ON b.book_id = l.book_id
```

Both return exactly 260 rows here. In practice RIGHT JOIN is rare – swapping table order and writing LEFT JOIN says the same thing, and most people find it easier to read.

FULL OUTER JOIN: Keep Everything

Keeps every row from **both** sides, padding with NULL on whichever side has no match.

```
SELECT b.title, l.loan_id
FROM Book b FULL OUTER JOIN Loan l ON b.book_id = l.book_id;
-- OUTER is optional
SELECT b.title, l.loan_id
FROM Book b FULL JOIN Loan l ON b.book_id = l.book_id;
```

Join type	Row count
LEFT JOIN (Book / Loan)	260
FULL OUTER JOIN (Book / Loan)	260

Identical here – Loan.book_id is NOT NULL and FK-enforced, so no loan can ever reference a nonexistent book. FULL OUTER only adds rows beyond LEFT JOIN when the **right** side can also have unmatched rows – foreign keys often rule that out, which is why FULL OUTER JOIN is the least-used of the four.

JOIN: a more complex case

Return all the names of the book on lone and their due date:

```
SELECT b.title, l.due_date
FROM book AS b JOIN loan AS l
  ON b.book_id = l.book_id
    AND l.return_date IS NULL;
```

It returns 63 records corresponding to the books that are on loan.

LEFT JOIN: a more complex case

Return all the names of the book on lone and their due date:

```
SELECT b.title, l.due_date
FROM book AS b LEFT JOIN loan AS l
  ON b.book_id = l.book_id
    AND l.return_date IS NULL;
```

It returns 99 records corresponding to the books that are on loan + titles of books that are not on a loan currently.

RIGHT JOIN: a more complex case

Return all the names of the book on lone and their due date:

```
SELECT b.title, l.due_date
FROM book AS b RIGHT JOIN loan AS l
  ON b.book_id = l.book_id
    AND l.return_date IS NULL;
```

It returns 258 records corresponding to the books that are on loan + all the retuned loans with `NULL` as book title.

FULL JOIN: a more complex case

Return all the names of the book on lone and their due date:

```
SELECT b.title, l.due_date
FROM book AS b FULL JOIN loan AS l
  ON b.book_id = l.book_id
    AND l.return_date IS NULL;
```

It returns 294 records corresponding to the books that are on loan

1. titles of books that are not on a loan currently
2. all the retuned loans with `NULL` as book title.

Self-Join: Joining a Table to Itself

A table can be joined to **itself**, using two aliases to tell the two "copies" apart. `Member.referred_by` points back at another `member_id` – who referred whom.

```
SELECT m.first_name, r.first_name AS referred_by_first
FROM Member m
LEFT JOIN Member r ON m.referred_by = r.member_id
ORDER BY m.member_id;
```

first_name	referred_by_first
Daniel	NULL
Freja	Daniel
Amalie	Peter

`m` and `r` are the **same table**, `Member`, aliased twice. Members 1–10 (founding members) have `referred_by IS NULL` – `LEFT JOIN` keeps them, with the referral column `NULL`.

CROSS JOIN: Every Combination

No matching condition at all – every row of the left table paired with every row of the right table.

```
SELECT * FROM Author, Book;
-- 42 authors x 80 books = 3360 rows
```

```
 count
-------
  3360
```

Almost never what you actually want. The classic **accidental** `CROSS JOIN` bug: writing `FROM table1, table2` (old comma-join syntax) and forgetting the `WHERE`/`ON` condition that ties them together – the query still runs, just returns a nonsensical Cartesian product instead of erroring out.

JOIN ... ON vs. JOIN ... USING

```sql
SELECT b.title, c.copy_number, c.status
FROM Book b JOIN Copy c ON b.book_id = c.book_id;

SELECT title, copy_number, status
FROM Book JOIN Copy USING (book_id);
```

Both verified to return the same 157 rows.

- `USING (col)` only works when the join column has the **same name** on both sides – it then collapses to one unqualified `book_id` column in the result, instead of `b.book_id`/`c.book_id`
- `ON` is fully general – required whenever column names differ, or the condition is not simple equality
- `ON` is the more common choice in practice, precisely because it always works

Multi-Table Joins: Three or More Tables

JOINs chain – each new `JOIN` clause adds one table, tied back to a table already in `FROM`.

```
SELECT m.first_name, b.title, a.last_name AS author_last
FROM Member m
JOIN Loan l ON m.member_id = l.member_id
JOIN Book b ON l.book_id = b.book_id
JOIN BookAuthor ba ON b.book_id = ba.book_id
JOIN Author a ON ba.author_id = a.author_id
ORDER BY m.member_id LIMIT 3;
```

first_name	title	author_last
Daniel	Nineteen Eighty-Four	Orwell
Daniel	Fahrenheit 451	Bradbury

Five tables, four JOINs, one query: “which member borrowed which book by which author.”

Multi-Table Joins + GROUP BY: A Worked Example

"Which authors have written books in more than one genre?" – straight from the course's own suggested in-class exercise, and a direct combination of last lecture's GROUP BY/HAVING with today's JOINs.

```sql
SELECT a.first_name, a.last_name, COUNT(DISTINCT g.genre) AS num_genres
FROM Author a
JOIN BookAuthor ba ON a.author_id = ba.author_id
JOIN BookGenre g ON ba.book_id = g.book_id
GROUP BY a.author_id, a.first_name, a.last_name
HAVING COUNT(DISTINCT g.genre) > 1
ORDER BY num_genres DESC;
```

first_name	last_name	num_genres
Daniel	Kahneman	4
Haruki	Murakami	3
George	Orwell	3

JOIN Condition Placement:

ON vs. WHERE

Setup: “All Members, Annotated With Overdue Loans”

Classic `LEFT JOIN` use case: show **every** member, with overdue-loan details attached **if they have one**.

Query A – filter condition inside `ON`:

```
SELECT m.member_id, m.first_name, l.loan_id, l.due_date
FROM Member m
LEFT JOIN Loan l
  ON m.member_id = l.member_id
  AND l.return_date IS NULL AND l.due_date < CURRENT_DATE
ORDER BY m.member_id;
```

member_id	first_name	loan_id	due_date
1	Daniel	NULL	NULL
5	Thomas	79	2026-07-24

56 rows total – all **50** members appear at least once. Overdue members get real values; everyone else gets `NULL`.

Query B: The Exact Same Predicate, Moved to `WHERE`

```sql
SELECT m.member_id, m.first_name, m.last_name, l.loan_id, l.due_date
FROM Member m
LEFT JOIN Loan l
  ON m.member_id = l.member_id
WHERE l.return_date IS NULL
  AND l.due_date < CURRENT_DATE
ORDER BY m.member_id;
```

Query	**Rows / distinct members**
A – predicate in `ON`	56 rows / **all 50** members
B – predicate in `WHERE`	25 rows / only **19** members

Same predicate, same tables, same `LEFT JOIN` keyword – but 31 members have silently vanished.

Why This Happens

- `ON` decides **which right-side rows get matched** – this runs **before** `NULL`-padding happens
- `WHERE` filters the **final** row set – this runs **after** `NULL`-padding
- For a member with no matching loan, `LEFT JOIN` still emits one row with every `l.*` column `NULL`
- Then `l.due_date < CURRENT_DATE` evaluates `NULL < CURRENT_DATE` – which is `NULL` (unknown), not `TRUE`
- `WHERE` discards any row that isn't `TRUE` – so every `NULL`-padded row is thrown away

Net effect: Query B's `LEFT JOIN` has been silently reduced to behaving like an `INNER JOIN`. The entire point of the `LEFT JOIN` – keep unmatched members – was undone in `WHERE`.

Caution: IS NULL Checks Are Different

Not every predicate is dangerous in WHERE – only ones like <, >, =, arithmetic, that are not NULL-preserving. IS NULL behaves the same in ON or WHERE, because NULL IS NULL evaluates to TRUE, not NULL.

```
SELECT COUNT(*) FROM Book b
LEFT JOIN Loan l ON b.book_id = l.book_id
WHERE l.return_date IS NULL;
-- 65 rows: 63 currently-outstanding loans + 2 never-borrowed books (NULL-padded)
```

Both the **genuinely** NULL rows (loans not yet returned) and the **padded** NULL rows (books with no loan at all) survive WHERE here – nothing is silently dropped. This is exactly why the ON/WHERE trap bites comparison and arithmetic predicates, not null-checks – and why testing the rule with the wrong kind of condition is the most common source of confusion.

Rule of thumb: filtering **which rows get matched** – put it in ON. Filtering the **final result** (with no outer-join rows to protect) – WHERE is fine.

Recap

JOIN type	Keeps
`INNER JOIN`	Only rows matching on both sides
`LEFT JOIN`	All left rows, `NULL`-padded right side if unmatched
`RIGHT JOIN`	All right rows, `NULL`-padded left side if unmatched
`FULL OUTER JOIN`	All rows from both sides
Self-join	Same table, two aliases
`CROSS JOIN`	Every combination – usually accidental

`ON` vs `USING`: `USING` only for identically-named columns. `ON` vs `WHERE`: comparison predicates in `WHERE` after an outer join silently undo it – put them in `ON` instead.

Subqueries in `WHERE`

Subqueries: A Query Inside a Query

A subquery is a complete `SELECT` nested inside another query. Three places one can appear:

- In `WHERE` – this block
- In `FROM` – a derived table (next block)
- In `SELECT` – a scalar value per row (next block)

Conceptually the inner query's result feeds the outer query – it can be thought of as running first.

WHERE ... IN (subquery)

```sql
SELECT first_name, last_name FROM Member
WHERE member_id IN (
  SELECT member_id FROM Loan
  WHERE return_date IS NULL AND due_date < CURRENT_DATE
);
```

first_name	last_name
Thomas	Sorensen
Mikkel	Pedersen
Oliver	Jensen

Same 19 distinct members as the `ON`-based `LEFT JOIN` two slides ago – “has at least one” reads cleanly as `IN`, and needs no `DISTINCT` the way a plain `JOIN` would to avoid repeated rows.

WHERE ... NOT IN (subquery)

```
SELECT COUNT(*) FROM Book
WHERE book_id NOT IN (SELECT book_id FROM BookGenre WHERE genre = 'Fantasy');
```

```
 count
-------
    65
```

65 of the library's 80 books are **not** tagged Fantasy. Reads naturally – but `NOT IN` has a sharp edge when the subquery's result can contain `NULL`, covered next.

Caution: `NOT IN` and `NULL`

```sql
SELECT COUNT(*) FROM Member
WHERE member_id NOT IN (SELECT referred_by FROM Member);
-- 0 rows -- for EVERY member, every time

SELECT COUNT(*) FROM Member
WHERE member_id NOT IN (SELECT referred_by FROM Member WHERE referred_by IS NOT
NULL);
-- 10 rows -- members 41-50, nobody's referrer
```

`referred_by` is `NULL` for the 10 founding members – so the first subquery's result list **contains** a `NULL`. Comparing anything with `NOT IN` against a list containing `NULL` makes the whole condition unknown for **every** row, so the query returns nothing at all. Filtering the subquery's `NULL`s out first fixes it. **Rule of thumb:** prefer `NOT EXISTS` over `NOT IN` unless the subquery's column is guaranteed `NOT NULL`.

WHERE EXISTS (subquery)

```sql
SELECT first_name, last_name FROM Member m
WHERE EXISTS (
  SELECT 1 FROM Loan l
  WHERE l.member_id = m.member_id AND l.return_date IS NULL
);
```

37 of 50 members currently have at least one unreturned loan. `EXISTS` cares only **whether** the inner query returns any row – the inner `SELECT 1` is idiomatic; the actual column selected is irrelevant.

WHERE NOT EXISTS (subquery): Books Never Borrowed

```
SELECT b.title FROM Book b
WHERE NOT EXISTS (SELECT 1 FROM Loan l WHERE l.book_id = b.book_id);
```

title
Guards! Guards!
Beloved

Exactly the 2 books identified earlier via `LEFT JOIN ... WHERE loan_id IS NULL` – same answer, two idioms. `NOT EXISTS` is often preferred: it doesn't require picking a specific column to test `IS NULL` on.

WHERE: Single-Value Comparison

A subquery that returns exactly one row, one column can be compared directly with `=`, `<`, `>`, etc.

```
SELECT title, publication_year FROM Book
WHERE publication_year > (SELECT AVG(publication_year) FROM Book);
```

title	publication_year
Dune	1965
One Hundred Years of Solitude	1967
A Wizard of Earthsea	1968

Average publication year across all 80 books: 1964.45. If the subquery ever returns **more than one row**, PostgreSQL raises `more than one row returned by a subquery used as an expression` – a runtime error, not a silently wrong answer.

Recap

Form	Reads as
`col IN (subquery)`	“matches any value the subquery returns”
`col NOT IN (subquery)`	“matches none – unsafe if subquery can return NULL”
`EXISTS (subquery)`	“at least one matching row exists”
`NOT EXISTS (subquery)`	“no matching row exists – the safe anti-join”
`col op (subquery)`	“compares against one single value”

Break

5-Minute Break

Stretch, grab water. Next: subqueries move out of `WHERE` and into `FROM` and `SELECT` – and we look at what a subquery costs when it depends on the outer query's current row.

Subqueries in `FROM` and `SELECT`

Subquery in FROM: Derived Tables

A subquery inside FROM acts as a temporary, named table for the rest of the query – an **inline view**.

```
SELECT * FROM (
  SELECT member_id, COUNT(*) AS num_loans FROM Loan GROUP BY member_id
) AS loan_counts
WHERE num_loans > 6
ORDER BY num_loans DESC;
```

member_id	num_loans
45	11
18	9
3	9

The derived table **must** have an alias (loan_counts). It lets the outer query filter on num_loans – a value that isn't a real column anywhere, only the result of an aggregate.

Scalar Subquery in SELECT

A subquery placed directly in the SELECT list must return exactly one value per outer row.

```
SELECT b.title,
  (SELECT COUNT(*) FROM Loan l WHERE l.book_id = b.book_id) AS times_borrowed
FROM Book b
ORDER BY times_borrowed DESC
LIMIT 5;
```

title	times_borrowed
Fahrenheit 451	13
Oryx and Crake	10
Never Let Me Go	9

This subquery references `b.book_id` from the **outer** row – it cannot run on its own. That makes it **correlated**, the topic of the next two slides.

Correlated Subqueries

- A correlated subquery references a column from the **outer** query's current row (`b.book_id` above)
- It cannot be run standalone – its result depends on which outer row is currently being evaluated
- Contrast with the `FROM`-clause derived table: that subquery ran **once**, independently, producing a fixed temporary table before the outer query even starts filtering it

```
-- correlated: depends on the outer row
(SELECT COUNT(*) FROM Loan l WHERE l.book_id = b.book_id)

-- NOT correlated: self-contained, same result regardless of outer row
(SELECT AVG(publication_year) FROM Book)
```

Correlated Subqueries: Performance Cost

- Conceptually, a correlated subquery re-runs **once per outer row** – 80 books here means the `times_borrowed` subquery is conceptually evaluated 80 times against `Loan`
- Real database engines often rewrite this internally (e.g. into a JOIN + `GROUP BY`) – but that optimization is not guaranteed for every query shape
- If a query with a correlated subquery in `WHERE` or `SELECT` feels slow on a large table, first check whether it can be rewritten as a `JOIN` or a `FROM`-clause derived table instead – both compute the aggregate **once**, not once per row

Recap

Form	Behaviour
Subquery in `FROM`	Runs once, independent, needs an alias, usable like a real table
Scalar subquery in `SELECT`	One value per outer row, often correlated
Correlated subquery	References the outer row; conceptually re-evaluated per row; watch performance

Set Operations and Recursive Queries

Set Operations: Combine Query Results

Set operations combine the **rows produced by two `SELECT` statements**.

```
query_a
UNION
query_b;
```

- `UNION` – rows from A or B, with duplicates removed
- `INTERSECT` – rows that appear in both A and B
- `EXCEPT` – rows in A but not in B
- `UNION ALL` – like `UNION`, but keeps duplicates

The two queries must return the same number of columns, with compatible data types in corresponding positions.

UNION: A or B

“Which books are tagged Fantasy **or** Science Fiction?”

```sql
SELECT book_id FROM BookGenre WHERE genre = 'Mystery'

UNION

SELECT book_id FROM BookGenre WHERE genre = 'Science Fiction';
```

The result contains each `book_id` at most once – even when the same book appears in **both** input queries.

Think of `UNION` as set addition:

$A \cup B$

It is useful when two different predicates describe alternative ways for a row to qualify. It results in 25 rows, 4 for `Mystery` and 22 for `Science Fiction`.

UNION ALL: Keep Duplicates

```sql
SELECT book_id FROM BookGenre WHERE genre = 'Mystery'

UNION ALL

SELECT book_id FROM BookGenre WHERE genre = 'Science Fiction';
```

- Every row from the first query is kept
- Every row from the second query is kept
- If the same `(book_id)` appears in both, it appears twice

`UNION` must remove duplicates, so it may need extra work. In this case, it results in 26 rows.

Operator	Meaning	Duplicates?
`UNION`	A or B	Removed
`UNION ALL`	A and B, concatenated	Kept

INTERSECT: What They Have in Common

“Which books are tagged **both** Fantasy and Science Fiction?”

```
SELECT book_id FROM BookGenre WHERE genre = 'Mystery'

INTERSECT

SELECT book_id FROM BookGenre WHERE genre = 'Science Fiction';
```

Only rows present in **both** result sets survive.

```
A ∩ B
```

A useful mental model: run the two SELECTs independently, then keep their common rows. Results in one row.

EXCEPT: In A, but Not in B

"Which books are tagged Fantasy but **not** Science Fiction?"

```
SELECT book_id FROM BookGenre WHERE genre = 'Mystery'

EXCEPT

SELECT book_id FROM BookGenre WHERE genre = 'Science Fiction';
```

A − B

EXCEPT is directional:

```
A EXCEPT B   -- rows only in A
B EXCEPT A   -- rows only in B
```

As with UNION and INTERSECT, duplicate rows in the inputs are removed in the final set result. EXCEPT ALL exists when multiplicities matter.

Set Operations: Precedence and `ORDER BY`

Set operations can be chained, but parentheses make intent explicit. (22 rows)

```
(
  SELECT book_id FROM BookGenre WHERE genre = 'Mystery'
  UNION
  SELECT book_id FROM BookGenre WHERE genre = 'Science Fiction'
)
EXCEPT
SELECT book_id FROM BookGenre WHERE genre = 'Comedy'
ORDER BY book_id;
```

- The three branches must have compatible column shapes
- `ORDER BY` applies to the **combined** result, so put it at the end
- Parentheses are useful when mixing different set operators
- Column names in the final result come from the first `SELECT`

Common Table Expressions

CTEs: `WITH ... AS`

A named, temporary result set defined once at the top of a query with `WITH`, then used like a real table in the rest of the statement – the same underlying idea as a `FROM`-clause derived table.

```sql
WITH loan_counts AS (
  SELECT member_id, COUNT(*) AS num_loans FROM Loan GROUP BY member_id
)
SELECT m.first_name, lc.num_loans
FROM Member m
JOIN loan_counts lc ON m.member_id = lc.member_id
WHERE lc.num_loans > 6
ORDER BY lc.num_loans DESC;
```

first_name	num_loans
Clara	11
Louise	9
Emma	9

CTEs vs. Derived Tables

- Same result as wrapping the same subquery in `FROM (...) AS loan_counts`
- The difference is **readability**: a CTE is named and defined once, up front, top-to-bottom – easier to follow once a query has several steps, and it can be referenced more than once later in the statement
- This is a preview only – multiple CTEs in one query and recursive CTEs (`WITH RECURSIVE`) are out of scope for this course, but worth knowing they exist for future queries with multi-step logic

Recursive Queries

WITH RECURSIVE: A Query That Refers to Itself - I

A recursive CTE defines a result set in terms of:

```
anchor rows
    ↓
recursive step
    ↓
more rows
    ↓
repeat until no new rows are produced
```

WITH RECURSIVE: A Query That Refers to Itself - II

General shape:

```
WITH RECURSIVE name AS (
  -- anchor member
  SELECT ...

  UNION ALL

  -- recursive member
  SELECT ...
  FROM name
  JOIN ...
)
SELECT * FROM name;
```

The `UNION ALL` is what accumulates the rows from the anchor and each recursive step.

Worked Example: Follow a Member’s Referral Chain - I

`Member.referred_by` points from a member to another member. Starting at one member, recursively follow the referrer upward.

```
WITH RECURSIVE referral_chain AS (
  -- anchor: start with one member
  SELECT member_id, first_name, last_name, referred_by, 0 AS depth FROM Member
    WHERE member_id = 25

  UNION ALL

  -- recursive step: find that member's referrer
  SELECT m.member_id, m.first_name, m.last_name, m.referred_by, rc.depth + 1
  FROM Member m JOIN referral_chain rc ON m.member_id = rc.referred_by
)
SELECT first_name, last_name, depth FROM referral_chain ORDER BY depth;
```

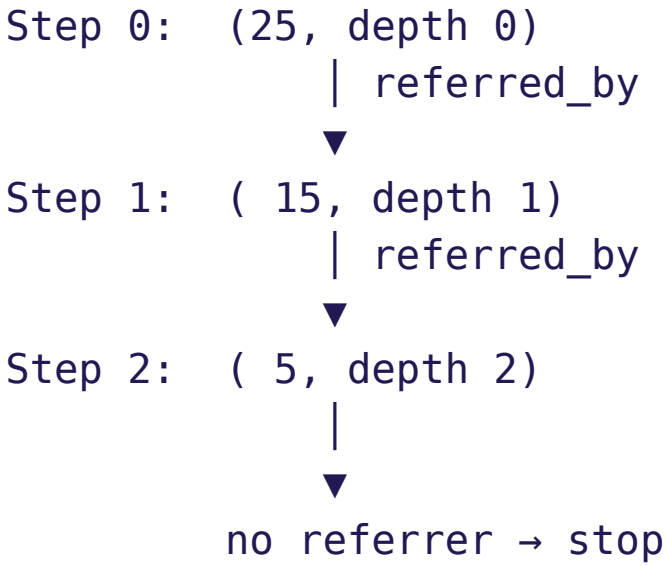
Worked Example: Follow a Member’s Referral Chain - II

first_name	last_name	depth
Clara	Sorensen	0
Sebastian	Petersen	1
Thomas	Sorensen	2

Replace `25` with the member whose referral chain you want to inspect.

How the Recursive CTE Unfolds

Suppose member 25 was referred by member 15, and member 15 was referred by member 5:



`WITH RECURSIVE`: Practical Guardrails

Recursive SQL is powerful, but make the stopping condition obvious.

```
WITH RECURSIVE numbers(n) AS (
  SELECT 1
  UNION ALL
  SELECT n + 1
  FROM numbers
  WHERE n < 10
)
SELECT n FROM numbers;
```

The `WHERE n < 10` clause is the termination condition. Without it, the example has no natural stopping point.

For graph-shaped data, also think about cycles. A node may eventually point back to a node already visited; in that case, track visited IDs (for example with an array path) or otherwise enforce a cycle guard.

Recap: Set Operations + Recursive CTEs

Construct	Think of it as…
`UNION`	combine rows from A and B, removing duplicates
`UNION ALL`	combine rows and keep duplicates
`INTERSECT`	keep rows common to A and B
`EXCEPT`	keep rows in A but not B
`WITH`	refers to a subclause
`WITH RECURSIVE`	build rows repeatedly from previous rows

Key distinction:

`JOIN` combines **columns across related rows**; set operations combine **result rows from separate queries**; recursive CTEs let a query repeatedly apply that idea to hierarchical or graph-shaped data.

Wrap-Up

Java Connection: The N+1 Query Problem

```
List<Member> members = getAllMembers();       // 1 query
for (Member m : members) {
    List<Loan> loans = getLoansForMember(m.getId());  // 1 query PER member!
}
```

50 members means 1 + 50 = 51 round-trips to the database, for data that a single `JOIN` could return in one round-trip.

```
SELECT m.*, l.*
FROM Member m LEFT JOIN Loan l ON m.member_id = l.member_id;
```

This exact tension – “loop in application code and query per row” vs. “one `JOIN`” – is the entire reason ORMs, covered later in the course, offer an explicit **eager** vs. **lazy** loading choice.

Summary

Today: all JOIN types (`INNER`, `LEFT`, `RIGHT`, `FULL OUTER`, self-join, `CROSS JOIN`), `ON` vs `USING`, the `ON`-vs-`WHERE` trap, subqueries in `WHERE`/`FROM`/`SELECT`, correlated subqueries and their cost, and a CTE preview.